

What is OOP?

Object-Oriented Programming is a programming approach that uses objects and classes to design and organize code.

Key Concepts of OOP

1. Class

- Blueprint of an object
- Example: `Car`, `Student`

2. Object

- Instance of a class
- Example: `myCar`, `student1`

3. Encapsulation

- Wrapping data + methods together
- Data hiding using `private/public`

4. Inheritance

- One class inherits properties of another
- Promotes code reuse

5. Polymorphism

- Same function → different behavior
- (Overloading & Overriding)

6. Abstraction

- Hiding complex details, showing only essentials

Advantages of OOP

- Code **reusability**
- Better **organization**
- Easy **maintenance**
- Improved **security** (data hiding)

What are the 4 pillars of OOP?

1. Encapsulation

🔒 *Wrapping data and methods into a single unit (class)*

- Protects data using **access modifiers** (`private`, `public`)

- Also called **data hiding**

✓ Example: Keeping variables private and accessing them via functions

◆ 2. Abstraction

☞ *Hiding complex implementation and showing only essential features*

- Focus on **what to do**, not **how to do**
- Achieved using **abstract classes** or **interfaces**

✓ Example: You drive a car without knowing how the engine works

◆ 3. Inheritance

☞ *One class acquires properties of another class*

- Promotes **code reuse**
- Types: single, multiple, multilevel, hierarchical

✓ Example: Dog inherits from Animal

◆ 4. Polymorphism

☞ *One function behaves differently in different situations*

- Types:
 - Compile-time → **Overloading**
 - Runtime → **Overriding**

✓ Example: Same function name, different behavior

What is a class and object?

What is a Class?

☞ A **class** is a **blueprint or template** used to create objects.

- It defines **variables (data members)** and **functions (member functions)**
- No memory is allocated until objects are created

★ Example:

```
class Student {
public:
    string name;
    int age;

    void display() {
        cout << name << " " << age;
    }
};
```

What is an Object?

☞ An **object** is an **instance of a class**.

- It represents a real-world entity
- Memory is allocated when an object is created
- Used to access class members

★ Example:

```
int main() {
    Student s1;    // object creation
    s1.name = "Kashish";
    s1.age = 20;
    s1.display();
}
```

Difference between class and structure in C++?

- **struct** → Simple data holder (like a form)
- **class** → Full system with control & security

What is constructor and destructor?

Constructor-

A **constructor** is a special member function that is **automatically called when an object is created**.

✔ Key Points:

- Same name as the class
- No return type (not even void)
- Used to **initialize objects**

★ Example:

```
#include <iostream>
using namespace std;

class Student {
public:
    Student() {    // constructor
        cout << "Constructor called" << endl;
    }
};

int main() {
    Student s1;    // constructor automatically called
}
```

Destructor-

A **destructor** is a special member function that is **automatically called when an object is destroyed**.

✔ Key Points:

- Same name as class with ~ (tilde)
- No return type
- Used to **free memory / cleanup**

★ Example:

```
#include <iostream>
using namespace std;

class Student {
public:
    ~Student() {    // destructor
        cout << "Destructor called" << endl;
    }
};

int main() {
    Student s1;    // destructor called when object goes out of scope
}
```

What are Access Specifiers?

☞ Access specifiers define **who can access class members**

Types:

- **public** → accessible everywhere
- **private** → accessible only inside class
- **protected** → accessible in class + derived class

Why Use Private Variables?

☞ Private variables are used for **data hiding and security**

✔ **Reasons:**

- Prevent **unauthorized access**
- Control data using **getter/setter functions**
- Improve **data integrity**
- Makes code **safe and maintainable**

What is Inheritance Ambiguity?

☞ **Ambiguity occurs when a derived class gets the same member from multiple base classes**, and the compiler gets confused about which one to use.

★ **Example:**

```
class A {
public:
    void show() {
        cout << "A";
    }
};

class B : public A {};
class C : public A {};

class D : public B, public C {
};

int main() {
    D obj;
    obj.show();    // ✕ Ambiguity
}
```

Solution to Ambiguity ★

☞ Use **scope resolution operator**

```
obj.B::show();
```

OR use **virtual inheritance**

Difference: Compile-Time vs Runtime Polymorphism

Feature	Compile-Time	Runtime
Binding	Early	Late
Method	Overloading	Overriding
Execution	Faster	Slightly slower
Example	Same function, different parameters	Virtual functions

What is Operator Overloading?

☞ **Operator overloading** means giving **custom meaning to operators** for user-defined objects.

★ Example:

```
obj1 + obj2;    // customized behavior
```

✓ Makes code more readable

What is Virtual Function?

☞ A **virtual function** is a function declared with `virtual` in base class and **redefined in derived class**.

✓ Ensures **correct function is called at runtime**

★ Example:

```
class A {
public:
    virtual void show() {
        cout << "A";
    }
};
```

What is Pure Virtual Function?

☞ A **pure virtual function** is a function with **no implementation** in base class.

★ Syntax:

```
virtual void show() = 0;
```

- ✓ Makes the class **abstract**

What is Abstract Class?

☞ A class that contains **at least one pure virtual function** is called an **abstract class**.

- ✓ Cannot create objects
- ✓ Used as a blueprint for derived classes

Constructor-

A **constructor** is a **special function in a class** that is **automatically called when an object is created**.

Key Points

- Same name as the class
- No return type (not even `void`)
- Automatically executed
- Used to **initialize values**

```
#include <iostream>
using namespace std;

class Student {
public:
    int age;

    // Constructor
    Student() {
        age = 18;
    }
};

int main() {
    Student s;    // constructor called automatically
    cout << s.age;
}
```

- ☞ Output: 18

Types of Constructors

1. Default Constructor

```
Student() { }
```

2. Parameterized Constructor

```
Student(int a) {  
    age = a;  
}
```

3. Copy Constructor

```
Student(Student &s) {  
    age = s.age;  
}
```

Yes 🐼 **you can initialize values without using a constructor** — but there are some differences you should understand.

1. Direct Initialization (Without Constructor)

✔ Method 1: Assign value after object creation

```
#include <iostream>  
using namespace std;  
  
class Student {  
public:  
    int age;  
};  
  
int main() {  
    Student s;  
    s.age = 18;    // direct assignment  
    cout << s.age;  
}
```

🔗 Works fine ✔

✔ Method 2: Using initialization at declaration (C++11)

```
class Student {  
public:  
    int age = 18;    // direct initialization  
};
```

💧 Then why use Constructor? ?

✗ Problem without constructor:

- You **may forget to initialize**
- No control over object creation
- Not good for complex logic

✔ Advantage of Constructor:

- Automatically initializes values
- Ensures object is always in valid state
- Can handle complex initialization

🔗 Example (Important Difference)

```
Student s;    // without constructor
cout << s.age;    // ✗ garbage value (uninitialized)
```

VS

```
Student s;    // with constructor
cout << s.age;    // ✔ always 18
```

📖 Easy Understanding

- Without constructor → **manual work**
- With constructor → **automatic + safe**

Key Rule (Remember this)

☞ **Same name + no return type → Constructor**

☞ **Same name + return type → Normal function**

If class and method have same name

✔ Case 1: No return type

```
class Student {
public:
    Student() {    // constructor
        cout << "Constructor";
    }
};
```

☞ This is a **constructor**✔ Automatically called when object is created

✗ Case 2: With return type

```
class Student {
public:
    void Student() {    // normal function
        cout << "Hello";
    }
};
```

☞ **No output**

☞ This is **NOT a constructor**.....✗ Not called automatically

✔ It is a **normal member function**

What will be output?

```
class A {
public:
    A() { cout << "A "; }
};

int main() {
    A a1, a2;
}
```

A) A B) A A C) Error D) No output

☞ ✔ **Answer: B**

Destructor is used to:

- A) Create object
- B) Delete object
- C) Initialize object
- D) Print object

👉 ✓ **Answer: B**